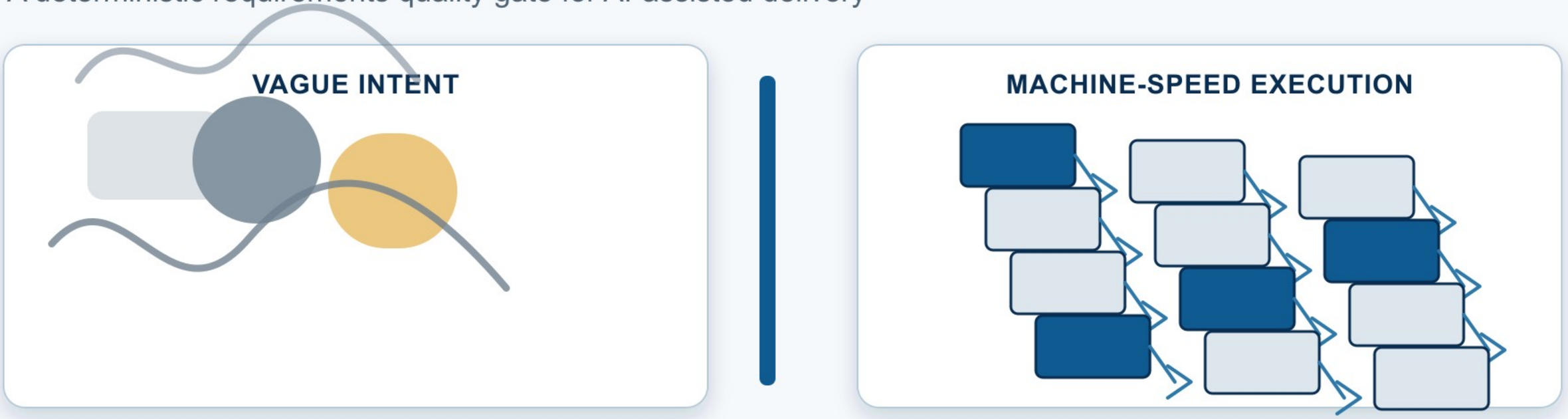
ALIBABA CLOUD x QODER

ClarityGate

A deterministic requirements quality gate for AI-assisted delivery



Bad requirements rarely fail loudly.
They become assumptions, rework, and code drift.

PRIMARY USERS

Business Analysts and Product Managers
Engineering teams
AI-assisted builders

Inspired by the Thoughtworks session on connecting requirements to runtime. No endorsement is implied.

The intent-to-code gap

The same uncertainty affects each handoff differently.

BA / PM	ENGINEERING TEAM	AI-ASSISTED BUILDER
TYPICAL INPUT Meetings, chats, BRDs, PRDs and stakeholder notes	**TYPICAL INPUT** Product and business requirements	**TYPICAL INPUT** Prompts and rough feature ideas
PRIMARY RISK Ambiguous intent reaches delivery as a handoff problem	**PRIMARY RISK** Implementation fills gaps with untracked assumptions	**PRIMARY RISK** The wrong product is generated faster

Ambiguity does not fail compilation.
It turns into assumptions, architecture, tests, and code.

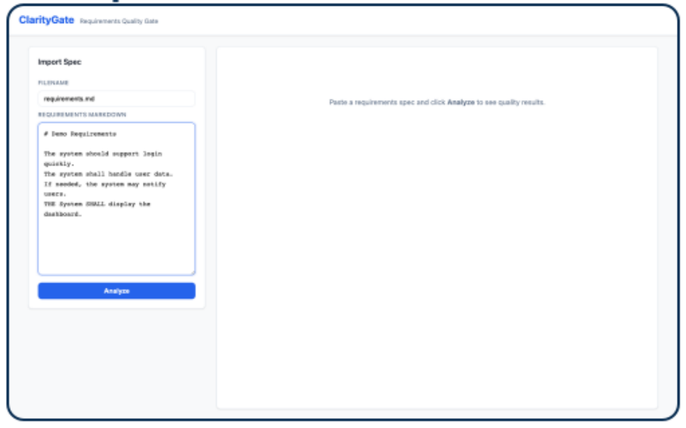
Terms worth challenging early: fast, user-friendly, as needed, immediately, handle, support.

Problem framing informed by the Thoughtworks requirements-to-runtime session and INCOSE writing guidance.

How ClarityGate works

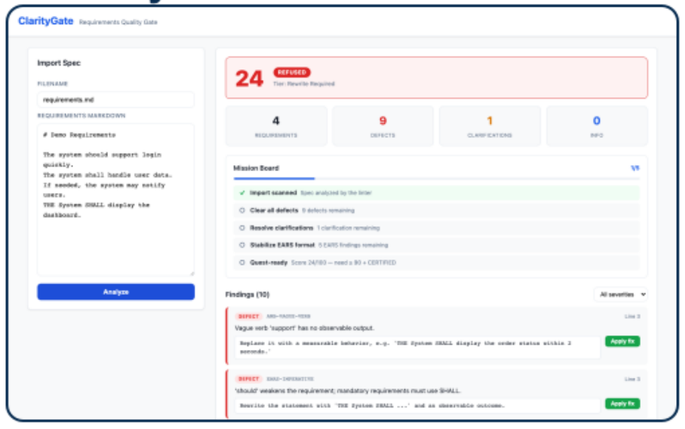
Real application screens from the working full-stack prototype

1. Import



Paste rough Markdown requirements.md

2. Analyze



Get a deterministic score, verdict, and severity totals

24 / 100 REFUSED

3. Review and fix

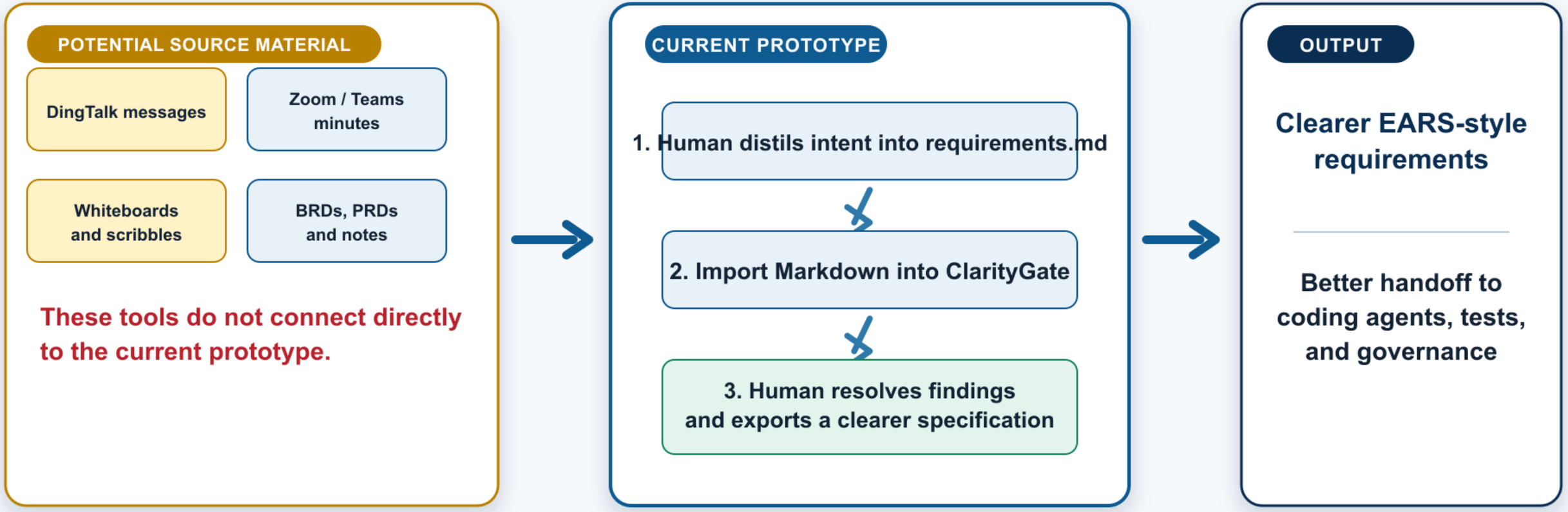
Inspect line-level findings and apply suggested rewrites

ClarityGate does not invent business decisions.

Deterministic checks expose uncertainty so the responsible human can resolve it.

From stakeholder noise to AI-ready specifications

Today is a manual Markdown workflow. Source integrations remain a future vision.



FUTURE VISION

Optional connectors could collect or convert content from collaboration tools. They are not implemented in this release.

Designed for trust, not magic

Technology stack

- Python standard-library deterministic linter
- FastAPI wrapper
- SQLite rewrite overlays
- React, Vite and TypeScript frontend
- Python unittest and equivalence tests

Engineering safeguards

- 36/36 frozen core tests passing
- 53/53 backend tests passing
- Same requirement, same result
- No auth, Firebase, external AI API, or cloud dependency
- Frozen core protected by review gates

BUILT WITH QODER

Quest Mode planning • Experts Mode execution • Reviewer gates • Rollback checkpoints

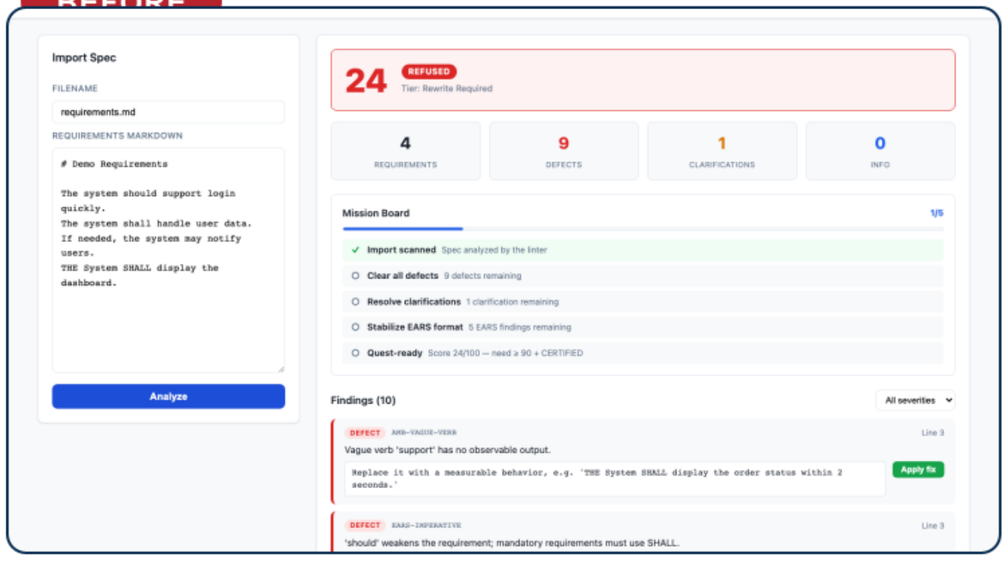
The build workflow protected the deterministic core while the full-stack wrapper grew around it.

github.com/yahgoo/ClarityGate

Visible proof: the score improves, and work remains

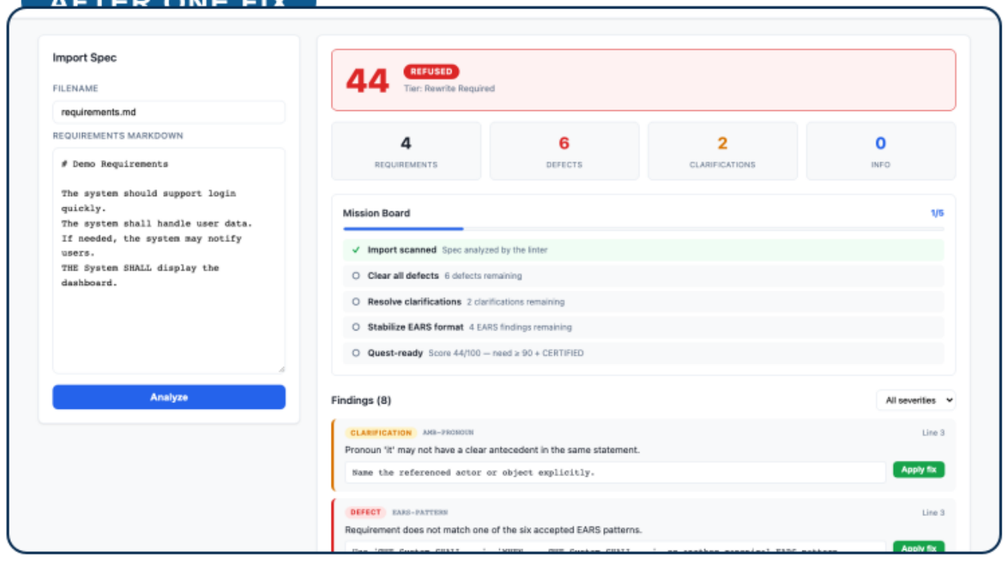
A deterministic rewrite changes the analysis immediately, without pretending the specification is finished.

BEFORE



REFUSED 24 / 100

AFTER ONE FIX



REFUSED 44 / 100

Mission Board tracks readiness

Markdown report supports handoff

Same Qoder.

Clearer specs in. Better software out.

Repository

https://github.com/yahgoo/ClarityGate